

Basic Security Document

Service Mobility Dashboard

Full-Stack Final Project

1. Purpose

This document describes the security controls implemented in the Service Mobility Dashboard, the trust boundaries of the current product, the way authentication and authorization are enforced, how sensitive configuration is handled, and the main security risks that remain for a future production version.

The dashboard contains internal operational service information. The security goal of the current version is therefore to ensure that only authenticated organizational users can access the shared service-task dataset and that normal application users cannot modify records mirrored from the organization's upstream operational source system.

2. Security Model and Data Ownership

service_tasks is a shared organizational operational dataset rather than a collection of records owned by individual dispatchers. In the current product, an authenticated dispatcher is intentionally allowed to read the same operational service-task information as another authenticated dispatcher.

Therefore, the main authorization boundary is currently between unauthenticated users and authenticated organizational users, not between individual users. If the product later supports multiple organizations, teams, or restricted dispatcher territories, the data model and RLS policies must add tenant/role-based scoping.

3. Authentication

Authentication is implemented with Supabase Auth using email and password credentials.

Login

- The login form submits credentials to the signIn Server Action.
- Both email and password are required before authentication is attempted.
- Supabase Auth validates the credentials.
- Failed authentication returns the generic message "Invalid email or password." rather than revealing whether a specific email address exists.
- There is no public self-registration flow; accounts are intended to represent authorized organizational employees.

Logout

signOut() terminates the Supabase session and redirects the user to /login. The implemented Playwright Secure Logout Flow verifies that the dashboard is inaccessible after logout and that direct navigation back to the protected route redirects to Login.

4. Request-Level Route Protection

src/proxy.ts protects application routes before the dashboard is rendered.

1. Supabase authentication cookies are read through the server client.
2. getClaims() is used to verify the authenticated JWT claims.
3. Unauthenticated requests to protected routes are redirected to /login.
4. The originally requested path and query string are preserved in redirectTo so a valid deep link can be restored after login.
5. Authenticated users who open /login are redirected back to the dashboard.

The proxy matcher excludes static assets and common image files so authentication redirects do not interfere with application JavaScript, CSS, or images.

5. Defense in Depth for Server Actions

The application does not rely only on the route proxy. Data-access Server Actions independently verify authentication before returning operational information.

getDailyTasks()

Calls Supabase getClaims() before querying service_tasks. If no valid authenticated claims are available, it returns an empty task array and a "Not authenticated" error instead of operational data.

getRecommendations()

Validates coordinates first and then performs its own Supabase getClaims() authentication check before retrieving future service-task records.

This second authorization check protects the data layer even if a route matcher is later changed or a Server Action is reused from another route.

6. Database Authorization with Row Level Security

Row Level Security is enabled on public.service_tasks. The database migration explicitly configures the read boundary:

```
ALTER TABLE public.service_tasks ENABLE ROW LEVEL SECURITY;

REVOKE SELECT ON public.service_tasks FROM anon;
GRANT SELECT ON public.service_tasks TO authenticated;
```

An explicit SELECT policy allows authenticated users to read the shared service-task dataset:

```
CREATE POLICY "authenticated users can read service tasks"
ON public.service_tasks
FOR SELECT
TO authenticated
USING (true);
```

This means that possession of the public Supabase anon key alone is not enough to read service_tasks. A valid authenticated session is required by the database policy.

7. Read-Only Protection for Mirrored Operational Data

The dashboard is intentionally a read-only decision-support layer. service_tasks is a one-way mirror of the organization's upstream operational source system or ERP (for example, SAP). Modifying the Supabase copy through normal application flows could create data that diverges from the authoritative source and could later be overwritten by synchronization.

The project therefore closes write access at the PostgreSQL privilege layer:

```
REVOKE INSERT, UPDATE, DELETE, TRUNCATE
ON public.service_tasks
FROM anon, authenticated;
```

The application also exposes no Server Action for creating, updating, or deleting service tasks. Read-only behavior is therefore enforced both in application design and at the database privilege layer.

Any production synchronization process with an upstream ERP, operational database, or other source system would need to use a privileged backend role such as service_role or another owner-level integration identity. Such a privileged key must never be exposed to the browser.

The repository also contains supabase/seed-e2e.sql for deterministic End-to-End test setup. This script is intended to be executed manually by an authorized database operator in the Supabase SQL Editor. It is not reachable through the application UI or normal authenticated application roles, and therefore does not weaken the read-only protection enforced for production users.

8. Input Validation

Date input

URL dates are validated as real ISO YYYY-MM-DD calendar dates. A string such as 2026-02-30 is rejected even though it matches the textual pattern, preventing malformed values from being used as normal date queries.

Recommendation coordinates

Latitude and longitude received by `getRecommendations()` are treated as untrusted client input. The Server Action accepts only finite values in the geographical ranges -90 to 90 latitude and -180 to 180 longitude before performing authentication, database access, or distance calculations.

Database coordinates

Coordinates read from `service_tasks` are validated independently before map routing and recommendation distance calculations, so malformed mirrored records do not reach Mapbox calculations.

Post-login redirect

`safeRedirectTo()` accepts only same-origin relative paths that begin with `/` and rejects protocol-relative values beginning with `//`. External redirect destinations therefore fall back to `/`, preventing an open-redirect phishing flow after a legitimate login.

External API responses

Mapbox optimization responses are validated before route geometry is accepted. Missing or malformed route data is treated as a route failure rather than being rendered as trusted geometry.

9. Session and Cookie Handling

The server-side Supabase client is created per request rather than being stored as a shared module-level session object. This avoids accidentally reusing one user's session context in another user's request.

The proxy refreshes Supabase authentication cookies when required and copies refreshed cookies onto redirect responses. Headers returned during the refresh flow are also preserved, preventing an authentication response from being treated like ordinary cacheable shared content.

10. Environment Variables and API Keys

Runtime configuration is stored in environment variables rather than hard-coded in the source repository.

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
NEXT_PUBLIC_MAPBOX_TOKEN
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY
```

The repository `.gitignore` contains `.env*`, so local environment files such as `.env.local` are excluded from Git tracking. Automated-test credentials are also read from environment variables rather than being hard-coded in Playwright tests.

Public versus secret keys

The variables prefixed with `NEXT_PUBLIC_` are intentionally available to browser JavaScript. They must therefore not be treated as secrets.

- The Supabase anon key is designed to be public; access to `service_tasks` is protected by authentication, RLS, and database privileges rather than by hiding the anon key.
- The Mapbox token is used client-side. The project code documents that it should be URL-restricted in the Mapbox account.
- The Google Places key is also used client-side and is therefore visible to users of the website. It should be restricted in Google Cloud to the required Places APIs and approved web origins/referrers. The repository itself cannot verify that cloud-side restriction.
- A Supabase `service_role` key or other privileged backend credential must never be stored in a `NEXT_PUBLIC_` variable or shipped to the browser.

11. Protection of External API Calls

Mapbox and Google Places are intentionally called from the browser because their current credentials are public client credentials. The application limits unnecessary Google Places traffic through a minimum three-

character query, approximately 300 ms debouncing, request cancellation, session tokens, and reuse of recent successful searches.

These measures improve efficiency but are not a substitute for provider-side key restrictions, quota limits, and usage monitoring. If a future external service requires a true secret key, the call must move behind a server-side Route Handler or Server Action rather than exposing the credential in `NEXT_PUBLIC_`.

12. Error Handling and Information Disclosure

User-facing errors are intentionally less detailed than server diagnostics. Authentication failures use a generic message, while Supabase query failures are logged server-side and returned to the UI as high-level operational errors. This reduces unnecessary disclosure of account or database details to the browser.

External route failures degrade gracefully: service-task markers remain available even when Mapbox cannot calculate a route. A failure in one vehicle route does not expose additional data or make the whole dashboard unavailable.

13. Local Browser Storage

The Recent Searches feature stores up to five previously selected addresses and coordinates in browser `localStorage` under `smd:recentAddressSearches`. No password, Supabase session token, or privileged API secret is intentionally stored by this feature.

However, searched addresses can still be operationally sensitive on a shared workstation. The current version does not automatically clear recent searches on logout or apply an expiry time. A stronger production version could clear the data on logout, store only less-sensitive display information, provide a user-visible clear-history control, or apply an expiration policy.

14. Current Authorization Limitation

The current RLS `SELECT` policy uses `USING (true)` for the authenticated role. This is correct for the present product model because dispatchers share one organizational operational dataset. It does not provide per-user, per-team, or per-organization row isolation.

If the system becomes multi-tenant or introduces restricted roles, `service_tasks` should include an organization/team ownership key and RLS policies should filter each authenticated identity to the rows it is permitted to view. Role-specific permissions could then distinguish dispatchers, managers, and administrators.

15. Security Verification Already Present

The current project includes security-relevant automated and database-level verification mechanisms.

- Playwright E2E-03 verifies secure logout and confirms that operational dashboard content cannot be accessed after the session ends; the full committed Playwright suite also exercises authenticated daily-planning and recommendation flows.
- The Server Actions contain explicit authentication checks rather than trusting the UI.
- The coordinate validation logic is covered by the committed Vitest unit-test suite.
- The RLS migration explicitly removes anonymous `SELECT` access and creates an authenticated `SELECT` policy.
- The read-only migration revokes `INSERT`, `UPDATE`, `DELETE`, and `TRUNCATE` from normal application roles and includes SQL verification queries for remaining policies/privileges.

16. Remaining Risks and Future Security Improvements

1. Verify and document production referrer/origin restrictions and quotas for the Google Places API key and Mapbox token.
2. Add explicit security response headers such as `Content-Security-Policy`, `Referrer-Policy`, and `Permissions-Policy` after defining the required Google, Mapbox, and Supabase origins. The current `next.config.ts` does not define custom security headers.
3. Add automated or repeatable database integration checks that prove anon cannot `SELECT` `service_tasks` and authenticated application users cannot `INSERT`, `UPDATE`, or `DELETE`.

4. Clear or expire Recent Searches on shared devices if searched addresses are considered sensitive operational data.
5. Introduce tenant/role-based RLS if the product expands beyond one shared organizational dataset.
6. Monitor failed login attempts and external API usage in production and add rate-limiting controls if real usage demonstrates a need.
7. Maintain dependency updates and run dependency/security scanning as part of the project release process.

17. Summary

The Service Mobility Dashboard currently applies basic security at multiple layers: Supabase email/password authentication, request-level route protection, repeated authentication checks inside data Server Actions, Row Level Security for database reads, explicit revocation of database writes, server-side validation of untrusted inputs, open-redirect protection, environment-based configuration, and generic user-facing authentication errors.

The most important remaining production improvements are provider-side restrictions for public browser API keys, explicit browser security headers, repeatable authorization tests, stronger handling of locally stored recent addresses on shared devices, and tenant/role-based RLS if the product grows beyond a single shared organizational environment.